

Binary Search & Heap

Binary search అంటే sorted array lo వెతకడం మాత్రమే కాదు
– "జవాబు మీద" search చేయడం.

ప్రతి problem కీ: ఏ **pattern** ఇది → ఎందుకు ఆ pattern → dry run → optimal
JavaScript code → complexity → edge cases. [DSA_Patterns_Telugu.pdf](#)
pattern-first దృష్టి; ఈ file ఆ patterns ని నిజమైన LeetCode problems మీద
అమలు చేస్తుంది.

ఈ document చదివిన తర్వాత Binary Search మరియు Heap problems మళ్ళీ నిన్ను భయపెట్టవు. ప్రతి problem కీ – **ఎలా ఆలోచించాలి** (naive నుండి insight వరకు), ఏ **pattern** వాడాలి, ఒక vivid real-life analogy, clean commented JavaScript solution, పెన్సిల్తో గీసినట్టు dry run (lo/hi/mid లేదా heap operations), complexity reasoning, గుర్తుంచుకోవాల్సినది, మరియు సాధారణ తప్పులు (off-by-one, mid overflow, empty) – అన్నీ ఉంటాయి. లక్ష్యం: **"ఒకసారి చదివితే మర్చిపోకూడదు."**

లక్ష్యం: DSA అస్సలు తెలియని person ని – ఎవరికైతే "sorted array లో వెతకడం" అంటే ఏమిటో, "heap" అంటే ఏమిటో కూడా తెలియదే – వాళ్ళని SSE (Senior Software Engineer) interview లో ఈ 11 problems confident గా solve చేసేలా తయారు చేయడం. మనం facts బట్టి పట్టడం కాదు – **ఎలా ఆలోచించాలో** నేర్చుకుంటాం. రెండు patterns లోతుగా అర్థమైతే, ఈ 11 మాత్రమే కాదు, వీటిలాంటి 100 problems కూడా solve చేయగలవు.

గమనిక: Big-O notation, arrays, recursion, time/space complexity, binary tree అంటే ఏమిటి – ఇలాంటి పునాదులు (fundamentals) [DSA_00_Foundations_Telugu.md](#) లో ఉన్నాయి. అవి ముందు చదివితే ఈ guide ఇంకా సులభంగా అర్థమవుతుంది. Solutions అన్నీ **JavaScript (ES2020+)** లో, runnable.

విషయ సూచిక (Table of Contents)

Pattern Primers (ముందు వీటిని చదువు – problems కి foundation):

- Pattern: Binary Search (invariant, off-by-one ని ఎలా తప్పించాలి, lower/upper bound, "binary search on answer", rotated & 2D)
- Pattern: Heap / Priority Queue (min vs max heap, top-K, two-heaps for median, reusable MinHeap/MaxHeap JS class)

Part 1 – Binary Search Problems

1. Search Insert Position (LeetCode #35) – Easy
2. Search a 2D Matrix (LeetCode #74) – Medium
3. Find Peak Element (LeetCode #162) – Medium
4. Search in Rotated Sorted Array (LeetCode #33) – Medium
5. Find First and Last Position of Element in Sorted Array (LeetCode #34) – Medium
6. Find Minimum in Rotated Sorted Array (LeetCode #153) – Medium
7. Median of Two Sorted Arrays (LeetCode #4) – Hard

Part 2 – Heap / Priority Queue Problems

8. Kth Largest Element in an Array (LeetCode #215) – Medium
9. IPO (LeetCode #502) – Hard
10. Find K Pairs with Smallest Sums (LeetCode #373) – Medium

Pattern: Binary Search

వివరణ

Binary Search అంటే – **sorted** (క్రమంలో అమర్చిన) data లో ఒక దాన్ని వెతకడానికి, ప్రతినారీ **మిగిలిన search space ని సగానికి తగ్గించడం**. ప్రతి అడుగులో మధ్య (mid) element ని చూసి, "నా target ఎడమ సగంలో ఉందా, కుడి సగంలో ఉందా?" అని నిర్ణయించి, సగం space ని పూర్తిగా వదిలేస్తాం. n elements ని $\log_2(n)$ అడుగుల్లో కుదించేస్తాం – 10 లక్షల elements ని కేవలం ~20 అడుగుల్లో!

కీలకమైన మూడు మాటలు:

- **lo, hi** – ప్రస్తుతం answer ఉండగల **range** (శోధించాల్సిన ప్రాంతం).
- **mid** – ఆ range మధ్య index. `mid = lo + Math.floor((hi - lo) / 2)`.
- **Invariant** – loop మొత్తం మీద ఎప్పుడూ నిజంగా ఉండే నియమం: "**answer ఎప్పుడూ ప్రస్తుత range లోపలే ఉంటుంది**". ఈ invariant ని strict గా పాటిస్తే off-by-one bugs మాయమవుతాయి.

అతి ముఖ్యమైన idea – "binary search on answer": Binary search అంటే కేవలం array లో వెతకడం కాదు. ఏదైనా **monotonic** (ఒక threshold దాటాక answer "yes $\rightarrow$ no" లేదా "no $\rightarrow$ yes" గా మారే) property ఉంటే, ఆ **answer values పరిధి మీద** binary search చేయవచ్చు. "కనిష్ఠ capacity ఎంత?", "కనిష్ఠ speed ఎంత?" లాంటి optimization problems ఇలా solve అవుతాయి – array sorted కాకపోయినా!

BINARY SEARCH • TEMPLATE మరియు "జవాబు మీద SEARCH"

INVARIANT – mid ని నమ్మవద్దు, పరిధిని నమ్మండి

1	3	5	7	9	11
0	1	2	3	4	5
lo					hi

while (lo < hi) – mid ని జవాబుగా return చేయకుండా పరిధిని కుదించడం

`if (condition(mid)) hi = mid; else lo = mid + 1; ← lo == hi అయినప్పుడు అదే జవాబు`

BINARY SEARCH ON ANSWER – array మీద కాదు, జవాబుల పరిధి మీద

false · false · false

true · true · true

↑ మొదటి true = జవాబు

షరతు – predicate MONOTONIC గా ఉండాలి

ఒకసారి true అయ్యాక ఆ తర్వాత ఎప్పుడూ true గానే ఉండాలి.

"10 kg మోయగలిగితే 9 kg కూడా మోయగలడు" – ఈ లక్షణం లేకపోతే binary search పని చేయదు.

రెండు పనులే: (1) పరిధి lo, hi ఏమిటి? (2) canDo(x) ఎలా check చేయాలి?

Constraint 10⁹ కనిపిస్తే binary search అనుమానించండి – $\log_2(10^9) \approx 30$ అడుగులు మాత్రమే. "కనిష్ఠ గరిష్ఠం" / "minimum capacity" / "ఎన్ని రోజుల్లో" అనే మాటలు దాదాపు ఎప్పుడూ binary-search-on-answer.

Real-life Scenario

Dictionary లో పదం వెతకడం. 1000 pages ఉన్న dictionary లో "Zebra" వెతకాలి. మొదటి page నుండి ఒక్కొక్కటి తిప్పుతూ వెళ్తావా? లేదు! సరిగ్గా **మధ్య page** తెరుస్తావు – "M" వచ్చింది. Z, M తర్వాత వస్తుంది కాబట్టి **మొదటి సగం (A-M) పూర్తిగా పక్కన పెట్టేస్తావు**. మిగిలిన సగం (M-Z) మధ్య తెరుస్తావు – "T". మళ్ళీ మొదటి సగం వదిలేస్తావు. ఇలా ప్రతిసారీ సగం తగ్గిస్తూ, కొన్ని అడుగుల్లోనే "Zebra" చేరుకుంటావు. ఇదే binary search – **sorted అయిన దానిలో, మధ్య చూసి, సగం తీసేయడం**.

ఎలా గుర్తించాలి (Recognition Signals)

- Input **sorted** గా ఉంటే (లేదా sort చేస్తే పని అవుతుంది అనిపిస్తే) – binary search మొదటి అనుమానం.
- Time constraint **$O(\log n)$** అని అడిగితే – దాదాపు ఖచ్చితంగా binary search.
- "మొదటి/చివరి occurrence", "insert position", "just greater/smaller" – lower/upper bound.
- **Rotated** sorted array, **2D sorted** matrix – binary search variants.
- "కనిష్ట/గరిష్ట **feasible** value ఎంత?" + ఒక `check(x)` monotonic function రాయగలిగితే – **binary search on answer**.

Reusable Templates (JavaScript)

రెండు template లు కంటతూ పెట్టుకో – 90% problems ఇవే.

```
// — Template A: Exact match (target ఉందా? index ఎక్కడ?) —————
// Range [lo, hi] – రెండు చివర్లు inclusive.
function binarySearch(nums, target) {
    let lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        const mid = lo + Math.floor((hi - lo) / 2); // range ఖాళీ అయ్యేదాకా
        if (nums[mid] === target) return mid;       // డిరికింది
        else if (nums[mid] < target) lo = mid + 1;   // target కుడి సగంలో
        else hi = mid - 1;                          // target ఎడమ సగంలో
    }
    return -1;                                       // లేదు
}

// — Template B: Lower bound (target >= అయ్యే మొదటి index) —————
// off-by-one కి అత్యంత safe. Range [lo, hi) – half-open, hi = n గమనించు.
function lowerBound(nums, target) {
    let lo = 0, hi = nums.length; // hi = n (insert-at-end కోసం)
    while (lo < hi) { // < (not <=)
        const mid = lo + Math.floor((hi - lo) / 2);
        if (nums[mid] < target) lo = mid + 1; // mid చాలదు → కుడికి
        else hi = mid; // mid candidate కావచ్చు → కోల్పోకు
    }
    return lo; // first index with nums[i] >= target (లేకపోతే n)
}
```

Template B invariant (ఇదే మ్యాజిక్): ఎప్పుడూ "answer `[lo, hi]` లోపల ఉంది" అని పట్టుకో.

`nums[mid] < target` అయితే mid ఖచ్చితంగా answer కాదు → `lo = mid + 1` (mid ని విడిచిపెట్టవచ్చు). లేకపోతే mid **answer కావచ్చు** → `hi = mid` (mid ని range లోనే ఉంచు, కానీ దాటి

పోకు). `lo < hi + hi = mid` కలిపి **infinite loop రాకుండా, off-by-one లేకుండా** చేస్తాయి. **upper bound** (target కంటే strictly పెద్ద మొదటి index) కావాలంటే condition ని `nums[mid] <= target` గా మార్చు.

```
// — Template C: Binary search on answer (monotonic feasibility) —
// "check(x) true అయ్యే కనిష్ట x ఎంత?" - false...false,true...true boundary.
function minFeasible(lo, hi, check) {
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (check(mid)) hi = mid;      // mid సరిపోతోంది → ఇంకా చిన్నది try చెయ్యి
    else lo = mid + 1;           // mid సరిపోలేదు → పెద్దది కావాలి
  }
  return lo;                    // సరిపోయే కనిష్ట value
}
```

Complexity

- **Time:** $O(\log n)$ – ప్రతి అడుగులో search space సగం అవుతుంది.
- **Space:** $O(1)$ iterative గా (పైన templates అన్నీ iterative). Recursive రాస్తే call stack $O(\log n)$.
- **Binary search on answer:** $O(\log(\text{range}) \times \text{check_cost})$. range = answer values పరిధి.

mid ని `lo + (hi - lo) / 2` గా ఎందుకు రాయాలి?

`(lo + hi) / 2` రాస్తే, పెద్ద arrays లో `lo + hi` **integer overflow** అవ్వొచ్చు (C/Java లో). JavaScript numbers 64-bit floats కాబట్టి practical గా overflow అరుదు – కానీ ఇది **universal మంచి అలవాటు**, interviewer expect చేస్తాడు. కాబట్టి ఎప్పుడూ `lo + Math.floor((hi - lo) / 2)` రాయు. `Math.floor` మర్చిపోకు – లేకపోతే mid float అయ్యి array index తప్పువుతుంది.

1. Search Insert Position (LeetCode #35) – Easy

సమస్య: ఒక **sorted** (ascending), **distinct** integers array `nums` మరియు ఒక `target` ఇస్తారు. target array లో ఉంటే దాని index return చెయ్యి. లేకపోతే, target ని sorted order లో ఎక్కడ insert చేస్తే array sorted గానే ఉంటుందో ఆ index return చెయ్యి. **$O(\log n)$** లో చెయ్యాలి.

Constraints: `1 <= nums.length <= 10^4`; `-10^4 <= nums[i]`, `target <= 10^4`; nums distinct, ascending sorted.

ఉదాహరణ:

```
nums = [1,3,5,6], target = 5 → 2   (5 already index 2 లో)
nums = [1,3,5,6], target = 2 → 1   (2 ని index 1 లో insert చేయాలి → [1,2,3,5,6])
nums = [1,3,5,6], target = 7 → 4   (అన్నిటికంటే పెద్దది → చివర్లో)
nums = [1,3,5,6], target = 0 → 0   (అన్నిటికంటే చిన్నది → మొదట్లో)
```

ఎలా ఆలోచించాలి: మొదట సహజంగా వచ్చే ఆలోచన – ఎడమ నుండి కుడికి నడుస్తూ, `nums[i] >= target` అయ్యే **మొదటి** index దగ్గర ఆగడం. అదే insert position. కానీ ఇది $O(n)$. Array **sorted** గా ఉంది,

పైగా **$O(\log n)$** అడిగారు → binary search వాడాలి. **కీలక insight:** "target ని ఎక్కడ insert చేయాలి?" అంటే " `nums[i] >= target` అయ్యే మొదటి index ఏది?" – ఇదే మన **lower bound** (Template B)! target array లో ఉంటే lower bound దాని exact index ని ఇస్తుంది; లేకపోతే insert position ని ఇస్తుంది. ఒకే code రెండు cases handle చేస్తుంది.

Brute Force / naive:

```
function searchInsertNaive(nums, target) {
  for (let i = 0; i < nums.length; i++) {
    if (nums[i] >= target) return i; // మొదటి >= target index
  }
  return nums.length; // అన్నీ చిన్నవి → చివర్లో
}
```

- **Time:** $O(n)$ – worst case మొత్తం array scan.
- **Space:** $O(1)$.
- **ఎందుకు సరిపోదు:** పని చేస్తుంది, కానీ array **sorted** అనే విలువైన property ని వాడుకోవడం లేదు. Problem $O(\log n)$ అడిగింది – linear scan ఆ constraint ని ఉల్లంఘిస్తుంది. Sorted data లో వెతకడానికి binary search ఉండగా $O(n)$ చేయడం waste.

Optimal Approach: Lower bound binary search (Template B). Half-open range `[lo, hi)` తో, `hi = n` (insertion చివర్లో కూడా జరగవచ్చు కాబట్టి). `nums[mid] < target` అయితే mid answer కాదు → `lo = mid + 1`. లేకపోతే mid candidate → `hi = mid`. Loop ఆగినప్పుడు `lo === hi` = insert position.

Solution (JavaScript):

```
function searchInsert(nums, target) {
  let lo = 0, hi = nums.length; // hi = n గమనించు (n-1 కాదు)
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (nums[mid] < target) {
      lo = mid + 1; // mid target కంటే చిన్నది → కుడికి జరుగు
    } else {
      hi = mid; // nums[mid] >= target → mid candidate, కోల్పోకు
    }
  }
  return lo; // first index with nums[i] >= target = insert position
}
```

Dry Run: `nums = [1,3,5,6], target = 2`

```
lo=0, hi=4
mid=0+(4-0)/2=2 → nums[2]=5 ; 5<2? లేదు → hi=2
lo=0, hi=2
mid=0+(2-0)/2=1 → nums[1]=3 ; 3<2? లేదు → hi=1
lo=0, hi=1
mid=0+(1-0)/2=0 → nums[0]=1 ; 1<2? అవును → lo=1
```

```
lo=1, hi=1 → loop ఆగింది (lo < hi false)
return lo = 1 ✓ (2 ని index 1 లో insert)
```

Complexity:

- **Time:** $O(\log n)$ – ప్రతి iteration search space సగం.
- **Space:** $O(1)$ – కొన్ని pointers మాత్రమే.

గుర్తుంచుకోవాల్సినది: "Insertion point" లేదా "first element $\geq$ target" అనే మాట వినగానే **lower bound** (Template B) అనుకో. `hi = n` పెట్టడం కీలకం – insertion array చివర్లో కూడా జరగవచ్చు. ఇదే lower-bound template Problem 5 (First & Last Position) కి కూడా వెన్నెముక. condition ని `<=` గా మారిస్తే **upper bound** వస్తుంది.

సాధారణ తప్పులు:

- **`hi = n - 1` పెట్టడం:** target అన్నిటికంటే పెద్దదైనప్పుడు (చివర్లో insert) చివరి index చేరలేక తప్పు answer వస్తుంది. Lower-bound లో `hi = n` ఉండాలి.
- **`hi = mid - 1` రాయడం:** Template B లో `hi = mid` (mid-1 కాదు) – mid answer కావచ్చు కాబట్టి range లోంచి తీసేయకూడదు. `mid - 1` పెడితే సరైన answer దాటిపోతావు.
- **`while (lo <= hi)` వాడటం:** Half-open range తో `<=` వాడితే infinite loop (`lo === hi` దగ్గర `mid=lo, hi=mid=lo` మారదు). Template B లో `<` మాత్రమే.
- **`Math.floor` మర్చిపోవడం:** mid float అయ్యి `nums[2.5]` = undefined → comparison తప్పువుతుంది.

2. Search a 2D Matrix (LeetCode #74) – Medium

సమస్య: $m \times n$ integer matrix ఇస్తారు. దీనికి రెండు properties: (1) ప్రతి **row** ఎడమ నుండి కుడికి **sorted**; (2) ప్రతి row యొక్క **మొదటి integer**, ముందు row యొక్క **చివరి integer** కంటే **పెద్దది**. ఒక `target` ఇస్తారు – అది matrix లో ఉందా (`true / false`) చెప్పాలి. $O(\log(m \cdot n))$ లో చెయ్యాలి.

Constraints: `1 <= m, n <= 100`; `-10^4 <= matrix[i][j], target <= 10^4`.

ఉదాహరణ:

```
matrix = [[1, 3, 5, 7],
          [10, 11, 16, 20],
          [23, 30, 34, 60]]
target = 3 → true
target = 13 → false
```

ఎలా ఆలోచించాలి: ఈ రెండు properties కలిపి చదువు: ప్రతి row sorted, పైగా ఒక row చివరి element < తర్వాతి row మొదటి element. అంటే – matrix ని **row by row వరుసగా జోడిస్తే**, అది ఒక **పూర్తిగా sorted 1D array** అవుతుంది! `[1, 3, 5, 7, 10, 11, 16, 20, 23, 30, 34, 60]`. Sorted 1D array లో వెతకడం = plain binary search (Template A). మిగిలిన ఒకే trick – 1D index `k` ని 2D coordinates (`row, col`) గా

మార్పుడం: `row = Math.floor(k / n)` , `col = k % n` ($n = \text{columns count}$). ఇలా actual గా కొత్త array కట్టకుండానే, virtual 1D array మీద binary search చేస్తాం – $O(1)$ extra space.

Brute Force / naive: ప్రతి cell scan – $O(m \cdot n)$. లేదా ప్రతి row లో binary search – $O(m \cdot \log n)$. రెండూ పని చేస్తాయి కానీ "whole matrix ఒకే sorted array" అనే బలమైన property ని పూర్తిగా వాడటం లేదు; $O(\log(m \cdot n))$ target ని అందుకోవు.

Optimal Approach: మొత్తం matrix ని $m \cdot n$ పొడవున్న ఒక virtual sorted array గా భావించి, దాని మీద **Template A** binary search. `mid` (1D index) ని `matrix[Math.floor(mid/n)][mid % n]` తో actual value గా మార్పు. $O(\log(m \cdot n))$.

Solution (JavaScript):

```
function searchMatrix(matrix, target) {
    const m = matrix.length, n = matrix[0].length;
    let lo = 0, hi = m * n - 1; // virtual 1D array [0 .. m*n-1]
    while (lo <= hi) {
        const mid = lo + Math.floor((hi - lo) / 2);
        // 1D index → 2D coordinates
        const val = matrix[Math.floor(mid / n)][mid % n];
        if (val === target) return true; // దొరికింది
        else if (val < target) lo = mid + 1; // కుడి సగం
        else hi = mid - 1; // ఎడమ సగం
    }
    return false; // లేదు
}
```

Dry Run: target = 3, $n = 4$, matrix పైనది ($m \cdot n = 12$)

```
lo=0, hi=11
mid=5 → row=Math.floor(5/4)=1, col=5%4=1 → matrix[1][1]=11 ; 11<3? లేదు → hi=4
lo=0, hi=4
mid=2 → row=0, col=2 → matrix[0][2]=5 ; 5<3? లేదు → hi=1
lo=0, hi=1
mid=0 → row=0, col=0 → matrix[0][0]=1 ; 1<3? అవును → lo=1
lo=1, hi=1
mid=1 → row=0, col=1 → matrix[0][1]=3 ; 3===3 → return true ✓
```

Complexity:

- **Time:** $O(\log(m \cdot n)) = O(\log m + \log n)$ – ఒకే binary search మొత్తం matrix మీద.
- **Space:** $O(1)$ – virtual 1D array; నిజంగా flatten చేయలేదు.

గుర్తుంచుకోవాల్సినది: "fully sorted 2D matrix" (row-wrap sorted) చూడగానే – **1D గా flatten చేసి binary search**. మ్యాజిక్ formula: `row = Math.floor(idx / cols)` , `col = idx % cols` . **జాగ్రత్త:** ఈ trick పని చేయాలంటే "ప్రతి row మొదటిది > ముందు row చివరిది" అనే **strong property** ఉండాలి. అది లేకపోతే (ప్రతి row & column మాత్రమే sorted – LeetCode #240 లాంటిది) ఈ approach తప్పు; అక్కడ top-right corner నుండి staircase search వాడాలి.

సాధారణ తప్పులు:

- **row/col formula లో n బదులు m వాడటం:** `mid / n`, `mid % n` – divisor ఎప్పుడూ **columns count (n)**, rows count కాదు. తారుమారు చేస్తే index తప్పువుతుంది.
- **hi = m * n పెట్టడం:** Template A inclusive range కాబట్టి `hi = m*n - 1`. `m*n` పెడితే `matrix[...]` out of bounds.
- **Empty matrix / empty row:** `matrix[0].length` access చేసేముందు matrix ఖాళీ కాదని నిర్ధారించు (constraints ఇక్కడ `m, n >= 1` కాబట్టి safe, కానీ generic గా చెక్ మంచిది).
- **#240 తో గందరగోళం:** "row-wise మరియు column-wise sorted" (కానీ wrap కాదు) అయితే ఈ code తప్పు. Property జాగ్రత్తగా చదువు.

3. Find Peak Element (LeetCode #162) – Medium

సమస్య: ఒక array `nums` ఇస్తారు. **Peak element** అంటే – దాని రెండు neighbors కంటే strictly పెద్దది (`nums[i-1] < nums[i] > nums[i+1]`). Array లో ఏదైనా ఒక peak యొక్క index return చెయ్యి (బహుళ peaks ఉంటే ఏదైనా సరే). ఊహించుకో: `nums[-1] = nums[n] = -∞` (సరిహద్దుల బయట negative infinity). పక్కపక్క elements ఎప్పుడూ సమానం కావు (`nums[i] != nums[i+1]`). **$O(\log n)$** లో చెయ్యాలి.

Constraints: `1 <= nums.length <= 1000`; `-231 <= nums[i] <= 231 - 1`; పక్కపక్క elements distinct.

ఉదాహరణ:

```
nums = [1,2,3,1]    → 2  (nums[2]=3, neighbors 2 & 1 కంటే పెద్దది)
nums = [1,2,1,3,5,6,4] → 5 (nums[5]=6 peak) లేదా 1 (nums[1]=2 peak) – ఏదైనా సరే
```

ఎలా ఆలోచించాలి: Array sorted కాదు – అయినా $O(\log n)$? ఇక్కడే మెదడు తిరుగుతుంది. Naive = linear scan, ప్రతి element neighbors తో పోల్చడం, $O(n)$. $O(\log n)$ కోసం binary search – కానీ target ఏది? **కీలక**

insight: mid దగ్గర **slope (ఏటవాలు)** చూడు. `nums[mid] < nums[mid+1]` అయితే – మనం

"ఎక్కుతున్నాం" (ascending). పైకి ఎక్కుతున్నప్పుడు, కుడివైపు ఖచ్చితంగా ఒక peak ఉంటుంది (ఎందుకంటే array చివర్లో $-\infty$ కి పడిపోతుంది కాబట్టి ఎక్కడో దిగాల్సిందే → అక్కడ peak). కాబట్టి peak **కుడివైపు** → `lo = mid + 1`. అలాగే `nums[mid] > nums[mid+1]` అయితే "దిగుతున్నాం" → peak `mid` లేదా **ఎడమవైపు** → `hi = mid`. ఇలా slope direction ప్రకారం ప్రతిసారీ సగం తీసేస్తాం. Sorted లేకపోయినా, boundary $-\infty$ guarantee వల్ల ఎప్పుడూ ఒక peak దొరుకుతుంది.

Brute Force / naive:

```
function findPeakNaive(nums) {
  for (let i = 0; i < nums.length; i++) {
    const left = i > 0 ? nums[i - 1] : -Infinity;
    const right = i < nums.length - 1 ? nums[i + 1] : -Infinity;
    if (nums[i] > left && nums[i] > right) return i;    // peak దొరికింది
  }
}
```

```
return -1; // constraints ప్రకారం ఇక్కడికి రాదు
}
```

- **Time:** $O(n)$ – worst case మొత్తం scan.
- **Space:** $O(1)$.
- **ఎందుకు సరిపోదు:** పని చేస్తుంది, కానీ $O(\log n)$ కావాలి. Slope idea వాడితే సగం space వదిలేయవచ్చు – linear scan అది వాడదు.

Optimal Approach: Slope ఆధారంగా binary search. `nums[mid] < nums[mid+1]` (ascending) → peak కుడివైపు, `lo = mid + 1`. లేకపోతే (descending లేదా mid itself peak) → `hi = mid`. Range `[lo, hi]` కుంచించుకుంటూ `lo === hi` అయినప్పుడు అదే peak.

Solution (JavaScript):

```
function findPeakElement(nums) {
  let lo = 0, hi = nums.length - 1;
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (nums[mid] < nums[mid + 1]) {
      // ఎక్కుతున్నాం → peak కుడివైపు ఉంది (mid కాదు)
      lo = mid + 1;
    } else {
      // దిగుతున్నాం (లేదా mid peak) → peak mid లేదా ఎడమవైపు
      hi = mid;
    }
  }
  return lo; // lo === hi → peak index
}
```

`mid + 1` ఎప్పుడూ valid – ఎందుకంటే `lo < hi` కాబట్టి `mid < hi`, అంటే `mid + 1 <= hi < n`. Out of bounds రాదు.

Dry Run: `nums = [1,2,1,3,5,6,4]`

```
lo=0, hi=6
mid=3 → nums[3]=3, nums[4]=5 ; 3<5 (ఎక్కుతున్నాం) → lo=4
lo=4, hi=6
mid=5 → nums[5]=6, nums[6]=4 ; 6<4? లేదు (దిగుతున్నాం) → hi=5
lo=4, hi=5
mid=4 → nums[4]=5, nums[5]=6 ; 5<6 (ఎక్కుతున్నాం) → lo=5
lo=5, hi=5 → loop ఆగింది
return 5 ✅ (nums[5]=6, neighbors 5 & 4 కంటే పెద్దది → peak)
```

Complexity:

- **Time:** $O(\log n)$ – sorted కాకపోయినా, ప్రతి అడుగులో సగం search space వదిలేస్తాం.
- **Space:** $O(1)$.

గుర్తుంచుకోవాల్సినది: "Binary search కేవలం sorted arrays కి మాత్రమే" అనే అపోహ తప్పు. ఒక **decision (slope/comparison)** ఆధారంగా "ఏ సగంలో answer ఖచ్చితంగా ఉంది" అని చెప్పగలిగితే చాలు. ఇక్కడ

కీలకం – mid ని mid+1 తో పోల్చి slope పట్టుకోవడం, boundary $-\infty$ guarantee. ఇదే idea "peak in mountain array" (#852), "single element in sorted array" (#540) లాంటి variants కి పని చేస్తుంది.

సాధారణ తప్పులు:

- `while (lo <= hi) + hi = mid`: infinite loop. Slope pattern లో ఎప్పుడూ `lo < hi + hi = mid` (mid-1 కాదు, ఎందుకంటే mid peak కావచ్చు).
- `nums[mid]` ని `nums[mid-1]` తో పోల్చడం: అప్పుడు mid-1 boundary (mid=0) దగ్గర out-of-bounds అవుతుంది. mid+1 తో పోల్చడం safe (`lo < hi` వల్ల ఎప్పుడూ valid).
- `>=` వాడటం: పక్కపక్క distinct కాబట్టి `<` చాలు; కానీ duplicates ఉన్న variant లో slope logic విఫలం.
- అన్ని peaks కావాలి అనుకోవడం: problem ఏదైనా ఒక peak అడిగింది. అన్ని peaks కావాలంటే $O(n)$ linear scan తప్పనిసరి – అప్పుడు binary search కుదరదు.

ROTATED SORTED ARRAY · ఒక సగం ఎప్పుడూ SORTED

ROTATED SORTED ARRAY – ఒక సగం ఎప్పుడూ sorted



ఎడమ సగం sorted అయితే

`nums[lo] ≤ nums[mid]`
target ఆ పరిధిలో ఉందా? → ఎడమకి
లేకపోతే → కుడికి

కుడి సగం sorted అయితే

`nums[mid] ≤ nums[hi]`
target ఆ పరిధిలో ఉందా? → కుడికి
లేకపోతే → ఎడమకి

కీలకమైన అంతర్దృష్టి

Rotated array ని మధ్యలో కట్టి – కనీసం ఒక సగం ఎప్పుడూ sorted గా ఉంటుంది.
ఆ sorted సగంలో target ఉందే లేదో $O(1)$ lo చెప్పొచ్చు (రెండు comparisons).
అందుకే binary search ఇక్కడ ఇంకా పని చేస్తుంది – $O(\log n)$.

4. Search in Rotated Sorted Array (LeetCode #33) – Medium

సమస్య: ఒక ascending sorted array ని ఏదో ఒక pivot దగ్గర rotate (తిప్పి) చేశారు – ఉదా.

`[0, 1, 2, 4, 5, 6, 7]` ని rotate చేస్తే `[4, 5, 6, 7, 0, 1, 2]`. ఈ rotated array `nums` (distinct values)

మరియు `target` ఇస్తారు. target ఉంటే దాని index, లేకపోతే `-1` return చెయ్యి. $O(\log n)$ లో చెయ్యాలి.

Constraints: `1 <= nums.length <= 5000`; `-10^4 <= nums[i]`, `target <= 10^4`; అన్ని values distinct; nums rotated sorted.

ఉదాహరణ:

```
nums = [4,5,6,7,0,1,2], target = 0 → 4
nums = [4,5,6,7,0,1,2], target = 3 → -1
nums = [1], target = 0 → -1
```

ఎలా ఆలోచించాలి: Array మొత్తం sorted కాదు, కానీ **సగం sorted** గా ఉంటుంది. `[4,5,6,7,0,1,2]` లో ఏ mid తీసుకున్నా – ఎడమ సగం `[lo..mid]` లేదా కుడి సగం `[mid..hi]` వాటిలో **కనీసం ఒకటి పూర్తిగా sorted** గా ఉంటుంది (rotation ఒకే చోట break చేస్తుంది కాబట్టి). ఏ సగం sorted అనేది `nums[lo] <= nums[mid]` చూసి తెలుసుకోవచ్చు: true అయితే ఎడమ సగం sorted. ఇప్పుడు – sorted సగంలో target ఉందా అనేది **రెండు వివర్ణ మధ్య range check** తో $O(1)$ లో చెప్పవచ్చు. target ఆ sorted సగంలో ఉంటే అటు వెళ్ళు, లేకపోతే మిగతా సగం వెళ్ళు. ఇలా rotation ఉన్నా binary search చేయగలం.

Brute Force / naive: Linear scan $O(n)$ – rotation ని పట్టించుకోకుండా ప్రతి element target తో పోల్చడం. పని చేస్తుంది కానీ $O(\log n)$ constraint ని ఉల్లంఘిస్తుంది; sorted-half structure ని వృథా చేస్తుంది.

Optimal Approach: Modified binary search. ప్రతి అడుగులో: (1) `nums[mid] === target` → return mid. (2) `nums[lo] <= nums[mid]` అయితే **ఎడమ సగం sorted** → target `[nums[lo], nums[mid])` range లో ఉంటే `hi = mid-1`, లేకపోతే `lo = mid+1`. (3) లేకపోతే **కుడి సగం sorted** → target `(nums[mid], nums[hi]]` range లో ఉంటే `lo = mid+1`, లేకపోతే `hi = mid-1`.

Solution (JavaScript):

```
function search(nums, target) {
    let lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        const mid = lo + Math.floor((hi - lo) / 2);
        if (nums[mid] === target) return mid;           // దొరికింది

        if (nums[lo] <= nums[mid]) {                    // — ఎడమ సగం sorted —
            if (nums[lo] <= target && target < nums[mid]) {
                hi = mid - 1;                           // target ఎడమ sorted సగంలో
            } else {
                lo = mid + 1;                             // లేకపోతే కుడివైపు
            }
        } else {                                       // — కుడి సగం sorted —
            if (nums[mid] < target && target <= nums[hi]) {
                lo = mid + 1;                             // target కుడి sorted సగంలో
            } else {
                hi = mid - 1;                             // లేకపోతే ఎడమవైపు
            }
        }
    }
    return -1;                                         // లేదు
}
```

Dry Run: `nums = [4,5,6,7,0,1,2]`, `target = 0`

```
lo=0, hi=6
mid=3 → nums[3]=7 ; 7===0? లేదు
nums[0]=4 <= 7 → ఎడమ సగం [4..7] sorted
4<=0 && 0<7 ? (0<=0 false) → else → lo=4
lo=4, hi=6
mid=5 → nums[5]=1 ; 1===0? లేదు
nums[4]=0 <= 1 → ఎడమ సగం [0..1] sorted
0<=0 && 0<1 ? అవును → hi=4
```

```
lo=4, hi=4
mid=4 → nums[4]=0 ; 0===0 → return 4 ✓
```

Complexity:

- **Time:** $O(\log n)$ – ప్రతి అడుగులో ఒక సగం పూర్తిగా వదిలేస్తాం.
- **Space:** $O(1)$.

గుర్తుంచుకోవాల్సినది: "Rotated sorted array" చూడగానే – "ఏ సగం sorted గా ఉంది?" అని అడుగు.

`nums[lo] <= nums[mid]` → ఎడమ sorted, లేకపోతే కుడి sorted. Sorted సగంలో target ఉందే లేదో range check తో నిర్ధారించి direction నిర్ణయించు. ఇదే backbone Problem 6 (Find Minimum) కి, "Search in Rotated Sorted Array II" (duplicates ఉన్న #81) కి.

సాధారణ తప్పులు:

- `nums[lo] < nums[mid]` (strict) వాడటం: `lo === mid` అయ్యే చిన్న ranges లో `<` fail అవుతుంది. `<=` వాడాలి (mid = lo అయినప్పుడు ఎడమ సగం trivially sorted).
- **Range boundaries లో strict/non-strict తప్పు:** ఎడమ sorted లో `target < nums[mid]` (strict, ఎందుకంటే `nums[mid] === target` ఇప్పటికే check అయింది), కానీ `nums[lo] <= target` (non-strict). ఈ boundaries తారుమారు చేస్తే edge cases fail.
- **Duplicates ఉంటే:** ఈ code distinct values కి. Duplicates ఉంటే (`[1,0,1,1,1]`) `nums[lo] === nums[mid]` అయినప్పుడు ఏ సగం sorted అని తేల్చలేం → worst case $O(n)$ కి పడిపోతుంది (`lo++` చేసి skip చేయాలి).
- `nums[mid] === target` check ని చివర్లో పెట్టడం: మొదట్లో పెట్టకపోతే logic సంక్లిష్టం అవుతుంది; ప్రతి iteration మొదట్లోనే check చెయ్యి.

5. Find First and Last Position of Element in Sorted Array (LeetCode #34) – Medium

సమస్య: ascending sorted array `nums` (duplicates ఉండవచ్చు) మరియు `target` ఇస్తారు. `target` యొక్క మొదటి మరియు చివరి occurrence indices ని `[first, last]` గా return చెయ్యి. `target` లేకపోతే `[-1, -1]`. $O(\log n)$ లో చెయ్యాలి.

Constraints: `0 <= nums.length <= 10^5`; `-10^9 <= nums[i], target <= 10^9`; `nums` ascending sorted.

ఉదాహరణ:

```
nums = [5,7,7,8,8,10], target = 8 → [3,4]
nums = [5,7,7,8,8,10], target = 6 → [-1,-1]
nums = [], target = 0 → [-1,-1]
```

ఎలా ఆలోచించాలి: Plain binary search target ని కనిపెడుతుంది, కానీ **ఎడో ఒక** occurrence దగ్గర ఆగిపోతుంది – first/last కాకపోవచ్చు. అక్కడి నుండి ఎడమ-కుడి linear గా విస్తరిస్తే, `[8,8,8,...,8]` లాంటి array కి $O(n)$ అవుతుంది (constraint break). **కీలక insight:** ఇది రెండు **boundary** problems: "target యొక్క మొదటి index" = **lower bound** (target $\geq$ అయ్యే మొదటి index); "target యొక్క చివరి index" = **upper bound - 1** (target కంటే strictly పెద్ద మొదటి index, దానికి ఒకటి తీసేయి). Problem 1 లో నేర్చుకున్న lower-bound template ని రెండుసార్లు వాడితే చాలు – ఒకసారి `target` కి, ఒకసారి `target+1` కి. రెండూ $O(\log n)$.

Brute Force / naive:

```
function searchRangeNaive(nums, target) {
  let first = -1, last = -1;
  for (let i = 0; i < nums.length; i++) {
    if (nums[i] === target) {
      if (first === -1) first = i; // మొదటిసారి
      last = i; // ప్రతిసారి update → చివరిది మిగులుతుంది
    }
  }
  return [first, last];
}
```

- **Time:** $O(n)$ – మొత్తం scan.
- **Space:** $O(1)$.
- **ఎందుకు సరిపోదు:** Sorted property ని పూర్తిగా వదిలేసింది. 10^5 elements, అన్నీ target అయితే – $O(n)$. $O(\log n)$ కావాలి కాబట్టి boundary binary search వాడాలి.

Optimal Approach: ఒక `lowerBound(nums, x)` helper (Template B) – `nums[i]  $\geq$  x` అయ్యే మొదటి index. అప్పుడు: `first = lowerBound(nums, target)`; అది `n` దాటితే లేదా `nums[first] !== target` అయితే target లేదు → `[-1, -1]`. లేకపోతే `last = lowerBound(nums, target + 1) - 1` (target కంటే పెద్ద మొదటిది, ఒకటి వెనక్కి).

Solution (JavaScript):

```
function searchRange(nums, target) {
  const first = lowerBound(nums, target);
  // target ఏ చోటా లేదా → not found
  if (first === nums.length || nums[first] !== target) return [-1, -1];
  const last = lowerBound(nums, target + 1) - 1; // (target కంటే పెద్ద మొదటిది) - 1
  return [first, last];
}

// nums[i]  $\geq$  target అయ్యే మొదటి index (లేకపోతే nums.length)
function lowerBound(nums, target) {
  let lo = 0, hi = nums.length; // half-open, hi = n
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (nums[mid] < target) lo = mid + 1; // mid చాలదు → కుడికి
    else hi = mid; // mid candidate → క్షేప్కు
  }
}
```

```
    return lo;
}
```

Dry Run: `nums = [5,7,7,8,8,10]`, `target = 8`

```
lowerBound(nums, 8):
  lo=0,hi=6 mid=3 nums[3]=8 ; 8<8? లేదు → hi=3
  lo=0,hi=3 mid=1 nums[1]=7 ; 7<8? అవును → lo=2
  lo=2,hi=3 mid=2 nums[2]=7 ; 7<8? అవును → lo=3
  lo=3,hi=3 ఆగింది → first = 3
  nums[3]=8 === target ✓ (not-found కాదు)

lowerBound(nums, 9):
  lo=0,hi=6 mid=3 nums[3]=8 ; 8<9? అవును → lo=4
  lo=4,hi=6 mid=5 nums[5]=10; 10<9? లేదు → hi=5
  lo=4,hi=5 mid=4 nums[4]=8 ; 8<9? అవును → lo=5
  lo=5,hi=5 ఆగింది → 5 ; last = 5 - 1 = 4
  return [3, 4] ✓
```

Complexity:

- **Time:** $O(\log n)$ – రెండు independent binary searches, ఒక్కొక్కటి $O(\log n)$.
- **Space:** $O(1)$.

గుర్తుంచుకోవాల్సినది: "First/last occurrence", "range of a value", "count of a value in sorted array" (= last - first + 1) – ఇవన్నీ **lower/upper bound** తో. గుర్తుపెట్టుకోవాల్సిన సూత్రం: **first = lowerBound(target)**, **last = lowerBound(target + 1) - 1**. ఒకే lower-bound helper తో రెండూ. ఇది count queries, `Math.floor` / `Math.ceil` style boundary problems అన్నిటికీ master key.

సాధారణ తప్పులు:

- **Not-found check మర్చిపోవడం:** `lowerBound` ఎప్పుడూ ఒక index ఇస్తుంది (target లేకపోయినా insertion point). కాబట్టి `first === n || nums[first] !== target` అని తప్పకుండా verify చెయ్యాలి, లేకపోతే తప్పు `[first, last]` వస్తుంది.
- **Empty array:** `nums = []` అయితే `lowerBound` 0 ఇస్తుంది, `first === nums.length` (`0 === 0`) true → సరిగ్గా `[-1, -1]`. కానీ మొదట not-found check లేకపోతే `nums[0]` = undefined access.
- **target + 1 overflow:** JS లో safe (numbers పెద్దవి), కానీ upper-bound ని ప్రత్యేకంగా రాయాలనుకుంటే condition `nums[mid] <= target` వాడు.
- **last కి +1 మర్చిపోవడం లేదా -1 పెట్టకపోవడం:** `lowerBound(target+1)` **target తర్వాత** మొదటి index; అసలు last occurrence దానికంటే ఒకటి ముందు → `-1` తప్పనిసరి.

6. Find Minimum in Rotated Sorted Array (LeetCode #153) – Medium

సమస్య: ascending sorted array ని `pivot` దగ్గర rotate చేశారు (distinct values). Rotated array `nums` ఇస్తారు – అందులో **కనిష్ఠ (minimum) element** return చెయ్యి. $O(\log n)$ లో చెయ్యాలి.

Constraints: $1 \leq \text{nums.length} \leq 5000$; $-5000 \leq \text{nums}[i] \leq 5000$; అన్నీ distinct; rotated sorted.

ఉదాహరణ:

```
nums = [3,4,5,1,2]      → 1
nums = [4,5,6,7,0,1,2]  → 0
nums = [11,13,15,17]    → 11 (అసలు rotate కాలేదు → మొదటిదే min)
```

ఎలా ఆలోచించాలి: Rotation వల్ల array లో ఒకే ఒక్క చోట "పడిపోవడం" ($\text{nums}[i] > \text{nums}[i+1]$)

ఉంటుంది – ఆ పడిపోయిన చోట minimum. Naive = linear scan $O(n)$. $O(\log n)$ కోసం binary search –

కీలక insight: `nums[mid]` ని array **చివరి element** `nums[hi]` తో పోల్చు. $\text{nums[mid]} > \text{nums[hi]}$ అయితే – mid ఇంకా "పెద్ద (ఎడమ) సగం" లో ఉంది, minimum ఖచ్చితంగా **mid తర్వాత కుడివైపు** → `lo = mid + 1`. $\text{nums[mid]} \leq \text{nums[hi]}$ అయితే – mid నుండి చివరిదాకా sorted, minimum **mid లేదా ఎడమవైపు** → `hi = mid` (mid ని కోల్పోకు, mid itself min కావచ్చు). ఎందుకు `nums[hi]` తో పోల్చాలి, `nums[lo]` తో కాదు? ఎందుకంటే `nums[hi]` ఎప్పుడూ minimum వైపు నిర్దిష్టమైన reference – rotate కానప్పుడు కూడా సరిగ్గా పని చేస్తుంది.

Brute Force / naive: `Math.min(...nums)` లేదా linear scan – $O(n)$. లేదా sort చేసి first తీసుకోవడం $O(n \log n)$ – ఇంకా worse. అన్నీ rotated structure ని వాడవు.

Optimal Approach: Binary search, `nums[mid]` vs `nums[hi]`. $\text{nums[mid]} > \text{nums[hi]}$ → `lo = mid + 1`; లేకపోతే `hi = mid`. `lo === hi` అయినప్పుడు అదే minimum index.

Solution (JavaScript):

```
function findMin(nums) {
  let lo = 0, hi = nums.length - 1;
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (nums[mid] > nums[hi]) {
      // mid "పెద్ద సగం" లో ఉంది → min ఖచ్చితంగా కుడివైపు
      lo = mid + 1;
    } else {
      // nums[mid] <= nums[hi] → min mid లేదా ఎడమవైపు (mid ని ఉంచు)
      hi = mid;
    }
  }
  return nums[lo]; // lo === hi → minimum
}
```

Dry Run: `nums = [4,5,6,7,0,1,2]`

```

lo=0, hi=6
mid=3 → nums[3]=7, nums[6]=2 ; 7>2 → lo=4
lo=4, hi=6
mid=5 → nums[5]=1, nums[6]=2 ; 1>2? లేదు → hi=5
lo=4, hi=5
mid=4 → nums[4]=0, nums[5]=1 ; 0>1? లేదు → hi=4
lo=4, hi=4 → loop ఆగింది
return nums[4] = 0 ✓

```

Rotate కానప్పుడు: `nums=[11,13,15,17]` . `mid=1` `nums[1]=13`, `nums[3]=17`; `13>17?` లేదు → `hi=1`.
`mid=0` `nums[0]=11`, `nums[1]=13`; `11>13?` లేదు → `hi=0`. `lo==hi=0` → `nums[0]=11` ✓.

Complexity:

- **Time:** $O(\log n)$.
- **Space:** $O(1)$.

గుర్తుంచుకోవాల్సినది: Rotated array లో minimum వెతకడానికి – `nums[mid]` ని `nums[hi]` తో పోల్చడం అనేదే master trick. `nums[mid] > nums[hi]` → min కుడివైపు (`lo = mid+1`); లేకపోతే `hi = mid`.
`nums[lo]` తో పోల్చవద్దు – rotate కాని array (`[1,2,3]`) లో `nums[mid] >= nums[lo]` ఎప్పుడూ true అయ్యే తప్పు దారి పడతావు. `nums[hi]` reference reliable. ఈ "min = rotation point" idea Problem 4 తో కలిపి – rotated array లో target ని "min కనిపెట్టి, పిమ్మట offset తో binary search" గా కూడా solve చేయవచ్చు.

సాధారణ తప్పులు:

- `nums[hi]` బదులు `nums[lo]` తో పోల్చడం: rotate కాని (fully sorted) array లో విఫలం. ఎప్పుడూ `nums[hi]` reference.
- `hi = mid - 1` రాయడం: mid itself minimum కావచ్చు కాబట్టి `hi = mid` (`mid-1` కాదు). `mid-1` పెడితే actual min దాటిపోతావు.
- `while (lo <= hi)`: `hi = mid` తో కలిపి infinite loop. `lo < hi` వాడు; loop ఆగినప్పుడు `lo == hi` = answer.
- **Duplicates ఉంటే (#154):** `nums[mid] === nums[hi]` అయినప్పుడు ఏ వైపు min అని తేల్చలేం → `hi--` చేసి safe గా కుంచించాలి; worst case $O(n)$.

7. Median of Two Sorted Arrays (LeetCode #4) – Hard

సమస్య: రెండు sorted arrays `nums1` (size `m`), `nums2` (size `n`) ఇస్తారు. రెండింటినీ కలిపితే వచ్చే combined sorted array యొక్క **median** (మధ్య విలువ) కనిపెట్టాలి. Total length బేసి అయితే మధ్య element; సరి అయితే మధ్య రెండింటి average. **$O(\log(m+n))$** లో చెయ్యాలి.

Constraints: `0 <= m, n <= 1000`; `1 <= m + n <= 2000`; `-10^6 <= nums[i] <= 10^6`; రెండూ ascending sorted.

ఉదాహరణ:

```
nums1 = [1,3], nums2 = [2]    → 2.0    (merged [1,2,3], మధ్య = 2)
nums1 = [1,2], nums2 = [3,4] → 2.5    (merged [1,2,3,4], (2+3)/2)
```

ఎలా ఆలోచించాలి: Naive = రెండు arrays ని merge చేసి (merge-sort లోని merge step) median తీసుకోవడం – $O(m+n)$. కానీ problem $O(\log(m+n))$ అడిగింది → merge చేయకూడదు! **కీలక insight:** Median అంటే – combined array ని రెండు సమాన సగాలుగా విభజించే విభజన రేఖ. ఎడమ సగంలో ప్రతిదీ కుడి సగంలో ప్రతిదాని కంటే $\leq$, రెండు సగాల్లో సమాన count. అలా విభజించాలంటే – `nums1` ని ఒక చోట `i` వద్ద, `nums2` ని ఒక చోట `j` వద్ద కోస్తే, ఎడమ భాగాల్లో మొత్తం half elements ఉండాలి (`j = half - i`). కాబట్టి `i` ఒక్కటి నిర్ణయిస్తే `j` దానంతట వస్తుంది. **చిన్న array మీద binary search చేసి సరైన i కనిపెడతాం** – condition: ఎడమ భాగపు గరిష్టాలు $\leq$ కుడి భాగపు కనిష్టాలు (`left1 <= right2 && left2 <= right1`). ఇదే "partition" idea.

Brute Force / naive:

```
function medianNaive(nums1, nums2) {
  const merged = [];
  let i = 0, j = 0;
  while (i < nums1.length && j < nums2.length) { // classic merge
    if (nums1[i] <= nums2[j]) merged.push(nums1[i++]);
    else merged.push(nums2[j++]);
  }
  while (i < nums1.length) merged.push(nums1[i++]);
  while (j < nums2.length) merged.push(nums2[j++]);
  const n = merged.length, mid = Math.floor(n / 2);
  return n % 2 ? merged[mid] : (merged[mid - 1] + merged[mid]) / 2;
}
```

- **Time:** $O(m + n)$ – merge మొత్తం.
- **Space:** $O(m + n)$ – merged array. (median position దాకా మాత్రమే merge చేస్తే space $O(1)$, కానీ time ఇంకా $O(m+n)$.)
- **ఎందుకు సరిపోదు:** పని చేస్తుంది, interview లో partial credit. కానీ $O(\log(m+n))$ target ని అందుకోదు – అదే ఈ problem ని "Hard" చేస్తుంది. Partition-based binary search మాత్రమే log target.

Optimal Approach: చిన్న array (`nums1`) మీద binary search – `i` = `nums1` నుండి ఎడమ భాగానికి తీసుకునే count (`0..m`). `j = half - i` (`half = ⌊(m+n+1)/2⌋` , +1 బేసి total ని ఎడమ వైపుకి పెడుతుంది). నాలుగు boundary values: `left1=nums1[i-1]` , `right1=nums1[i]` , `left2=nums2[j-1]` , `right2=nums2[j]` (సరిహద్దుల బయట $\pm\text{Infinity}$). **సరైన partition:** `left1 <= right2 && left2 <= right1` . `left1 > right2` అయితే `i` తగ్గించు (`hi = i-1`); లేకపోతే పెంచు (`lo = i+1`).

Solution (JavaScript):

```
function findMedianSortedArrays(nums1, nums2) {
  // ఎప్పుడూ చిన్న array మీద binary search → O(log(min(m,n)))
  if (nums1.length > nums2.length) return findMedianSortedArrays(nums2, nums1);

  const m = nums1.length, n = nums2.length, total = m + n;
  const half = Math.floor((total + 1) / 2); // ఎడమ భాగంలో ఉండాలైన count
  let lo = 0, hi = m; // i పరిధి [0, m]
```

```

while (lo <= hi) {
    const i = lo + Math.floor((hi - lo) / 2); // nums1 నుండి ఎడమకి i elements
    const j = half - i;                      // nums2 నుండి ఎడమకి j elements

    // సరిహద్దుల బయట ±Infinity (partition చెల్లుబాటు కోసం)
    const left1 = i > 0 ? nums1[i - 1] : -Infinity;
    const right1 = i < m ? nums1[i] : Infinity;
    const left2 = j > 0 ? nums2[j - 1] : -Infinity;
    const right2 = j < n ? nums2[j] : Infinity;

    if (left1 <= right2 && left2 <= right1) { // — సరైన partition —
        if (total % 2 === 1) return Math.max(left1, left2); // బేసి
        return (Math.max(left1, left2) + Math.min(right1, right2)) / 2; // సరి
    } else if (left1 > right2) {
        hi = i - 1; // nums1 నుండి ఎక్కువ తీసుకున్నాం → i తగ్గించు
    } else {
        lo = i + 1; // nums1 నుండి తక్కువ తీసుకున్నాం → i పెంచు
    }
}
return -1; // valid inputs కి ఇక్కడికి రాదు
}

```

Dry Run: `nums1 = [1,3], nums2 = [2] → swap (nums1 పెద్దది) → nums1 = [2], nums2 = [1,3]`

```

m=1, n=2, total=3, half=[4/2]=2
lo=0, hi=1
i=0, j=2-0=2
left1=-Inf, right1=nums1[0]=2, left2=nums2[1]=3, right2=+Inf
left1<=right2 (-Inf<=Inf ✓) && left2<=right1 (3<=2 ✗) → else-if left1>right2? -
Inf>Inf ✗ → else → lo=1
lo=1, hi=1
i=1, j=2-1=1
left1=nums1[0]=2, right1=+Inf, left2=nums2[0]=1, right2=nums2[1]=3
left1<=right2 (2<=3 ✓) && left2<=right1 (1<=Inf ✓) → సరైన partition!
total%2=1 → return max(left1,left2)=max(2,1)=2 ✓

```

Complexity:

- **Time:** $O(\log(\min(m, n)))$ – చిన్న array మీద మాత్రమే binary search.
- **Space:** $O(1)$ – merge చేయలేదు, కేవలం pointers.

గుర్తుంచుకోవాల్సినది: ఇది binary search యొక్క **అత్యున్నత రూపం** – array **values** మీద కాదు, **partition position** మీద search. కారణం: (1) చిన్న array మీద search, (2) `half = [(total+1)/2]`, `j = half - i`, (3) సరిహద్దుల బయట `±Infinity` (edge cases ని if-else లేకుండా handle చేసే అందమైన trick), (4) condition `left1<=right2 && left2<=right1`. ఈ partition idea "kth element of two sorted arrays" కి కూడా generalize అవుతుంది.

సాధారణ తప్పులు:

- **చిన్న array మీద search చేయకపోవడం:** `nums1` పెద్దదైతే `j = half - i` negative లేదా `n` దాటవచ్చు → out of bounds. మొదట swap తప్పనిసరి.

- **$\pm$ Infinity sentinels మర్చిపోవడం:** $i=0$ లేదా $i=m$ (ఒక array పూర్తిగా ఒక వైపు) edge cases లో $\text{nums1}[-1] / \text{nums1}[m]$ undefined $\rightarrow$ comparison NaN. $-\text{Infinity} / +\text{Infinity}$ ఈ boundary లను శుభ్రంగా పరిష్కరిస్తాయి.
- **half లో +1 మర్చిపోవడం:** $\lfloor \text{total}/2 \rfloor$ పెడితే బేసి total కి median తప్పవుతుంది. $\lfloor (\text{total}+1)/2 \rfloor$ ఎడమ భాగానికి ఎక్కువ (బేసి case లో మధ్య element ఎడమ వైపుకి) $\rightarrow$ $\text{Math.max}(\text{left1}, \text{left2})$ నేరుగా median.
- **సరి/బేసి branch తారుమారు:** బేసి $\rightarrow \text{max}(\text{left1}, \text{left2})$; సరి $\rightarrow (\text{max}(\text{lefts}) + \text{min}(\text{rights})) / 2$. తారుమారు చేస్తే తప్పు.

Pattern: Heap / Priority Queue

వివరణ

Heap (లేదా **Priority Queue**) అంటే – ఒక special data structure, దీనిలో ఎప్పుడూ **అత్యధిక ప్రాధాన్యత (highest/lowest priority) element** ని $O(1)$ లో చూడవచ్చు, $O(\log n)$ లో తీయవచ్చు, $O(\log n)$ లో కొత్తది చేర్చవచ్చు. Sorted గా ఉంచడం అవసరం లేదు – కేవలం "టాప్ ఎవరు?" అనేది క్షణంలో తెలియడం చాలు.

రెండు రకాలు:

- **Min-Heap** – అతి చిన్న element ఎప్పుడూ top (root) లో. $\text{peek}()$ = minimum.
- **Max-Heap** – అతి పెద్ద element ఎప్పుడూ top లో. $\text{peek}()$ = maximum.

లోపల ఇది ఒక **complete binary tree**, కానీ array తో represent చేస్తారు: index i కి – parent $\lfloor (i-1)/2 \rfloor$, children $2i+1$ & $2i+2$. **Heap property:** ప్రతి parent తన children కంటే (min-heap లో) చిన్నది. కొత్త element చేర్చినప్పుడు చివర్లో పెట్టి **sift-up** (పైకి పోల్చుకుంటూ జరపడం); top తీసినప్పుడు చివరిదాన్ని top కి తెచ్చి **sift-down** (కిందికి జార్చడం). రెండూ $\text{tree height} = O(\log n)$.

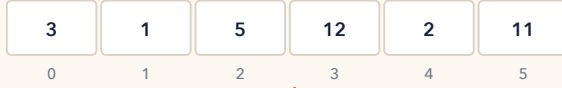
Top-K problems కి heap ఎందుకు perfect: "అతి పెద్ద/చిన్న k elements" కావాలంటే – మొత్తం sort ($O(n \log n)$) అవసరం లేదు. **Size k heap** maintain చేస్తే $O(n \log k)$. k చిన్నదైతే ఇది చాలా వేగం.

Two-heaps for median: stream లో median కావాలంటే – చిన్న సగం ఒక **max-heap** లో, పెద్ద సగం ఒక **min-heap** లో ఉంచి, రెండింటి sizes balance చేస్తే – median ఎప్పుడూ రెండు tops దగ్గరే (Problem 11).

JavaScript లో built-in Priority Queue లేదు! (Java `PriorityQueue`, C++ `priority_queue`, Python `heapq` ఉన్నాయి – JS కి లేదు.) కాబట్టి interview లో **మనమే ఒక heap class రాయాలి**. కింద ఇచ్చిన reusable class ని కంఠతా పెట్టుకో – Problems 8-11 అన్నీ దీనిమీదే ఆధారపడతాయి.

HEAP · SIZE-K కిటికీ

TOP-K · అతి పెద్ద 3 కావాలంటే – MIN-heap ని size 3 తో ఉంచడం



Min-heap · size = 3

పైన ఉన్నది వేటిలో అతి చిన్నది

కొత్తది > heap top → pop చేసి push

లేకపోతే వదిలేయడం

Sort చేస్తే

$O(n \log n)$ – అన్నిటిని క్రమపరుస్తాం,
కానీ కావలసినవి K మాత్రమే.
Streaming data కి sort అసలు కుదరదు.

Heap తో

$O(n \log K)$ – K చిన్నదైతే చాలా వేగం.
Memory కూడా $O(K)$ మాత్రమే.

ఎందుకు min-heap: అతి పెద్ద K ని ఉంచాలంటే – బయటికి పంపాల్సినది వాటిలో అతి చిన్నది

Real-life Scenario

Hospital emergency room (triage). రోగులు వచ్చిన వరుసలో కాదు – **తీవ్రత (severity)** ప్రకారం చూస్తారు. గుండెపోటు వచ్చినవాడు, జలుబు వచ్చినవాడి కంటే ముందు. కొత్త రోగి వచ్చినప్పుడు అతన్ని severity ప్రకారం సరైన చోట చేర్చుతారు ($O(\log n)$), తర్వాతి రోగిని పిలవాలంటే **అత్యంత తీవ్రమైనవాడు** వెంటనే తెలుస్తాడు ($O(1)$). మొత్తం రోగులను severity ప్రకారం పూర్తిగా sort చేయాల్సిన అవసరం లేదు – "టాప్ ఎవరు" మాత్రమే ముఖ్యం. అదే max-heap. Min-heap అంటే – "అతి తక్కువ severity" ముందు (ఉదా. tokens పంచడం).

ఎలా గుర్తించాలి (Recognition Signals)

- "Kth largest / smallest", "top K", "K closest", "K most frequent" – top-K, heap of size k.
- "Running / streaming median", "data stream" – two heaps.
- "ప్రతిసారీ అతి చిన్న/పెద్ద దాన్ని తీసి process చెయ్యి, కొత్తవి చేర్చు" – Dijkstra, merge k sorted lists, task scheduling.
- "Greedy + ప్రతి అడుగులో current best కావాలి" (Problem 9 IPO లాంటివి).
- Sort చేస్తే పని అవుతుంది కానీ **అన్నీ కావు, top కొన్ని మాత్రమే** అవసరం అనిపిస్తే – heap $O(n \log k) <$ sort $O(n \log n)$.

Reusable Heap Class (JavaScript)

```
// — Generic binary heap (comparator ఆధారంగా) —————  
// compare(a, b) < 0 అంటే a కి ఎక్కువ priority (top కి దగ్గర).  
class Heap {  
  constructor(compare) {  
    this.data = [];  
    this.compare = compare;  
  }  
  size() { return this.data.length; }  
  peek() { return this.data[0]; } // top element (తీయకుండా)  
  isEmpty() { return this.data.length === 0; }  
  
  push(val) {  
    this.data.push(val);  
    // చివర్లో పెట్టు
```

```

        this._siftUp(this.data.length - 1); // సరైన స్థానానికి పైకి జరుపు
    }

    pop() {
        const top = this.data[0];
        const last = this.data.pop(); // చివరి element తీసేయి
        if (this.data.length > 0) {
            this.data[0] = last; // దాన్ని root కి పెట్టి
            this._siftDown(0); // కిందికి జార్చు
        }
        return top; // పాత top ని return
    }

    _siftUp(i) {
        while (i > 0) {
            const parent = (i - 1) >> 1; // [(i-1)/2]
            if (this.compare(this.data[i], this.data[parent]) < 0) {
                [this.data[i], this.data[parent]] = [this.data[parent], this.data[i]];
                i = parent;
            } else break; // heap property సరిపోయింది
        }
    }

    _siftDown(i) {
        const n = this.data.length;
        while (true) {
            let best = i; // parent, children లో top candidate
            const l = 2 * i + 1, r = 2 * i + 2;
            if (l < n && this.compare(this.data[l], this.data[best]) < 0) best = l;
            if (r < n && this.compare(this.data[r], this.data[best]) < 0) best = r;
            if (best === i) break; // ఇక జరపాల్సింది లేదు
            [this.data[i], this.data[best]] = [this.data[best], this.data[i]];
            i = best;
        }
    }
}

// — MinHeap: చిన్న value top కి (numbers కి default; objects కి compare ఇవ్వ)
class MinHeap extends Heap {
    constructor(compare = (a, b) => a - b) { super(compare); }
}

// — MaxHeap: పెద్ద value top కి — compare ని తిప్పేస్తే చాలు.
class MaxHeap extends Heap {
    constructor(compare = (a, b) => a - b) { super((a, b) => compare(b, a)); }
}

```

వాడకం (usage):

```

const min = new MinHeap(); // min.push(5); min.push(1); min.push(3);
min.peek(); // 1 (అతి చిన్నది) min.pop(); // 1 తీసేసింది

const max = new MaxHeap(); // max.push(5); max.push(1); max.push(3);
max.peek(); // 5 (అతి పెద్దది)

// objects కి custom comparator: capital చిన్నది top కి
const byCap = new MinHeap((a, b) => a.cap - b.cap);

```

```
byCap.push({ cap: 2, prof: 9 }); byCap.push({ cap: 0, prof: 1 });
byCap.peak(); // { cap: 0, prof: 1 }
```

Complexity

Operation	Time
<code>peek()</code> (top చూడు)	$O(1)$
<code>push(x)</code> (చేర్చు)	$O(\log n)$ – sift-up tree height
<code>pop()</code> (top తీయి)	$O(\log n)$ – sift-down tree height
<code>size()</code> / <code>isEmpty()</code>	$O(1)$
n elements నుండి heap కట్టడం (ఒక్కొక్కటి push)	$O(n \log n)$
Heapify (array ని in-place heap చేయడం – bottom-up sift-down)	$O(n)$

- **Space:** $O(n)$ – n elements store చేయడానికి.
- **Top-K:** size-k heap $\rightarrow O(n \log k)$ time, $O(k)$ space.

గుర్తుంచుకోవాల్సిన కీలక ఉపాయం (counter-intuitive!)

"**k-th LARGEST**" కావాలంటే $\rightarrow$ **size-k MIN-heap** వాడు (max కాదు!). ఎందుకంటే – min-heap top ఎప్పుడూ "ఇప్పటిదాకా టాప్-k లో అతి బలహీనుడు". కొత్తది వాడి కంటే పెద్దదైతే, వాడిని తీసేసి కొత్తది చేర్చు. చివర్లో heap లో top-k largest ఉంటాయి, వాటిలో **అతి చిన్నదే = k-th largest** = `peek()`. అలాగే "**k-th smallest**" $\rightarrow$ **size-k MAX-heap**. ఈ తిరకాసు మొదట గందరగోళంగా ఉంటుంది కానీ Problem 8 తో స్పష్టమవుతుంది.

8. Kth Largest Element in an Array (LeetCode #215) – Medium

సమస్య: ఒక integer array `nums` మరియు integer `k` ఇస్తారు. **sorted order లో k-th largest** element return చెయ్యి. గమనిక: ఇది **distinct** k-th కాదు – duplicates లెక్కలోకి వస్తాయి (sorted array లో k-th పెద్ద స్థానంలో ఉన్నది). పూర్తిగా sort చేయకుండా solve చేయగలవా?

Constraints: $1 \leq k \leq \text{nums.length} \leq 10^5$; $-10^4 \leq \text{nums}[i] \leq 10^4$.

ఉదాహరణ:

```
nums = [3,2,1,5,6,4], k = 2      → 5   (sorted [6,5,4,3,2,1], 2nd = 5)
nums = [3,2,3,1,2,4,5,5,6], k = 4 → 4   (duplicates లెక్కలోకి; 4th largest = 4)
```

ఎలా ఆలోచించాలి: సులభమైన ఆలోచన – descending sort చేసి `nums[k-1]` తీసుకోవడం. $O(n \log n)$. పని చేస్తుంది కానీ మనకు **మొత్తం క్రమం అవసరం లేదు** – కేవలం k-th largest చాలు. ఇక్కడే heap

మెరుస్తుంది. **కీలక insight (heap primer లో చెప్పిన తిరకాసు):** "k-th largest" కావాలంటే **size-k MIN-heap** వాడు. array మీద నడుస్తూ ప్రతి element ని heap లో push చెయ్యి; heap size k దాటితే `pop()` (min తీసేయి). చివర్లో heap లో **అతి పెద్ద k elements** మిగులుతాయి, వాటిలో **అతి చిన్నది (heap top) = k-th largest**. మొత్తం sort అవసరం లేదు, size-k heap చాలు.

Brute Force / naive:

```
function findKthLargestSort(nums, k) {  
  nums.sort((a, b) => b - a); // descending sort  
  return nums[k - 1]; // k-th largest  
}
```

- **Time:** $O(n \log n)$ – పూర్తి sort.
- **Space:** $O(\log n)$ – $O(n)$ sort implementation ని బట్టి.
- **ఎందుకు సరిపోదు:** పని చేస్తుంది, interview లో acceptable కూడా. కానీ మనం **మొత్తం order** అడగలేదు – k-th మాత్రమే. $k \ll n$ అయితే heap $O(n \log k)$ చాలా వేగం. Follow-up "streaming / very large n, small k" కి sort సరిపోదు.

Optimal Approach: Size-k **MinHeap** (heap primer నుండి). ప్రతి num ని push; `size() > k` అయితే `pop()`. చివర్లో `peek()` = k-th largest. Time $O(n \log k)$, space $O(k)$.

Solution (JavaScript):

```
// ↑ heap primer లోని MinHeap class ఇక్కడ available అనుకుందాం.  
function findKthLargest(nums, k) {  
  const heap = new MinHeap(); // చిన్నది top → టాప్-k లో బలహీనుడు top లో  
  for (const num of nums) {  
    heap.push(num);  
    if (heap.size() > k) {  
      heap.pop(); // అతి చిన్నది తీసేయి → పెద్ద k మిగులుతాయి  
    }  
  }  
  return heap.peek(); // heap లో అతి చిన్నది = k-th largest  
}
```

Dry Run: `nums = [3,2,1,5,6,4]`, `k = 2` (size-2 min-heap, top = min)

push 3	→ heap {3}	size 1
push 2	→ heap {2,3}	size 2
push 1	→ heap {1,2,3}	size 3 > 2 → pop min(1) → {2,3}
push 5	→ heap {2,3,5}	size 3 > 2 → pop min(2) → {3,5}
push 6	→ heap {3,5,6}	size 3 > 2 → pop min(3) → {5,6}
push 4	→ heap {4,5,6}	size 3 > 2 → pop min(4) → {5,6}
peek() = 5	✓ (heap లో {5,6}, top = min = 5 = 2nd largest)	

Complexity:

- **Time:** $O(n \log k)$ – n elements, ఒక్కొక్క push/pop $O(\log k)$ (heap size $\leq k$).

- **Space:** $O(k)$ – heap లో ఎప్పుడూ $\leq k$ elements.

గుర్తుంచుకోవాల్సినది: "k-th largest $\rightarrow$ size-k MIN-heap", "k-th smallest $\rightarrow$ size-k MAX-heap" – ఈ రెండు వాక్యాలు కంఠతా. Heap top ఎప్పుడూ "టాప్-k లో అతి బలహీనుడు (గేటు కీపర్)" – కొత్తది వాడిని ఓడిస్తేనే లోపలికి. ఇదే idea "K closest points" (#973), "K most frequent" (#347), "top K frequent words" కి. **Quickselect** (average $O(n)$) alternative ఉంది కానీ heap streaming-friendly & సులభం.

సాధారణ తప్పులు:

- **k-th largest కి max-heap వాడటం:** పని చేస్తుంది కానీ మొత్తం n push + $(k-1)$ pop $\rightarrow O(n \log n)$, heap size $O(n)$. Size-k min-heap యొక్క $O(n \log k)$ benefit పోతుంది. తిరకాసు గుర్తుంచుకో.
- **size() > k బదులు $\geq k$ వాడటం:** $\geq k$ పెడితే size k-1 కి పడిపోయి, చివర్లో top తప్పు element అవుతుంది. $> k$ అయినప్పుడే pop.
- **Distinct k-th అనుకోవడం:** duplicates లెక్కలోకి వస్తాయి. distinct k-th కావాలంటే Set వాడాలి (వేరే problem).
- **Empty heap మీద peek:** `k <= nums.length` guaranteed కాబట్టి safe, కానీ generic గా empty check మంచిది.

MERGE K SORTED · MIN-HEAP

MERGE K SORTED LISTS – min-heap తే



అమాయక విధానం

అన్నీ కలిపి sort చేయడం
 $O(N \log N)$, N = మొత్తం nodes
 Streaming data కి కుదరదు

Heap తే

$O(N \log k)$ – k lists సంఖ్య
 Heap lo ఎప్పుడూ k elements మాత్రమే
 k చిన్నదైతే గణనీయమైన తేడా

ప్రత్యామ్నాయం – divide and conquer

Lists ని జతలుగా merge చేస్తూ పోవడం – అదే $O(N \log k)$, కానీ heap అవసరం లేదు.
 Merge sort యొక్క merge step ని $\log k$ సార్లు వాడటం.

9. IPO (LeetCode #502) – Hard

సమస్య: నీ దగ్గర మొదట w capital (డబ్బు) ఉంది. **గరిష్టంగా** k projects ఎంచుకోవచ్చు. ప్రతి project i కి – దాన్ని మొదలుపెట్టడానికి `capital[i]` కావాలి (ప్రస్తుత capital $\geq$ దీనికి ఉండాలి), పూర్తయితే `profits[i]` **లాభం** (అది capital కి కలుస్తుంది). ఒక project ఒకసారే చేయగలవు. **k projects తర్వాత గరిష్ట capital** ఎంత? (ఇది "IPO ముందు లాభం maximize చేయడం" scenario.)

Constraints: $1 \leq k \leq 10^5$; $0 \leq w \leq 10^9$; $1 \leq n \leq 10^5$; `profits[i]`, `capital[i]` పెద్ద range.

ఉదాహరణ:

```
k=2, w=0, profits=[1,2,3], capital=[0,1,1] → 4
w=0: capital≤0 project = #0 (profit 1). చెయ్యి → w=1.
w=1: capital≤1 projects = #1(prof2), #2(prof3). ఎక్కువ లాభం #2 → w=1+3=4.
```

ఎలా ఆలోచించాలి: ప్రతి అడుగులో ఏ project ఎంచుకోవాలి? **Greedy insight:** ప్రస్తుత capital తో **afford చేయగల** projects లో, **అత్యధిక profit** ఉన్నదాన్ని ఎంచుకోవడమే optimal (profit ఎప్పుడూ positive కాబట్టి, ఎక్కువ profit → ఎక్కువ future capital → ఎక్కువ options). కానీ ప్రతి అడుగులో "afford చేయగలవి ఏవి?" మారుతుంది (capital పెరిగేకొద్దీ కొత్త projects unlock). Naive గా ప్రతిసారీ అన్ని projects scan చేస్తే $O(k \cdot n)$.

రెండు heaps తో వేగం: (1) projects ని **capital ప్రకారం MIN-heap** లో ఉంచు (చౌకవి top). (2) ప్రస్తుత capital తో afford చేయగల అన్నిటినీ **profit ప్రకారం MAX-heap** లోకి తరలించు. అప్పుడు max-heap top = ఇప్పుడు చేయదగిన అత్యధిక-profit project. దాన్ని తీసి capital పెంచు. capital పెరిగినప్పుడు ఇంకొన్ని unlock → మళ్ళీ min-heap నుండి తరలించు.

Brute Force / naive: ప్రతి k iterations లో, అన్ని n projects ని scan చేసి "affordable వాటిలో max profit" కనిపెట్టి, ఆ project ని mark చేయడం. **Time:** $O(k \cdot n) = 10^5 \times 10^5 = 10^{10}$ → చాలా నెమ్మది, TLE. **Space:** $O(n)$ done-flag. Afford check + max-profit ప్రతిసారీ మొదటినుండి చేయడం waste – heaps ఆ పునరావృత పనిని $O(\log n)$ కి తగ్గిస్తాయి.

Optimal Approach: రెండు heaps. **byCapital** = MinHeap on capital (అన్ని projects init). **byProfit** = MaxHeap on profit . k times: (1) **byCapital.peek().cap ≤ w** ఉన్నంతవరకు pop చేసి **byProfit** లోకి push (unlock). (2) **byProfit** ఖాళీ అయితే break (ఇక ఏదీ afford చేయలేం). (3) లేకపోతే **w += byProfit.pop()** (అత్యధిక profit చేపట్టు). చివర్లో **w**.

Solution (JavaScript):

```
// ↑ heap primer నుండి MinHeap, MaxHeap.
function findMaximizedCapital(k, w, profits, capital) {
    const n = profits.length;

    // capital తక్కువన్నవి top → చౌక projects ముందు unlock అవుతాయి
    const byCapital = new MinHeap((a, b) => a.cap - b.cap);
    for (let i = 0; i < n; i++) {
        byCapital.push({ cap: capital[i], prof: profits[i] });
    }

    // afford చేయగల వాటిలో profit ఎక్కువన్నది top
    const byProfit = new MaxHeap(); // numbers (profit) – default comparator

    for (let i = 0; i < k; i++) {
        // ప్రస్తుత capital w తో afford చేయగల అన్ని projects ని unlock చెయ్యి
        while (byCapital.size() > 0 && byCapital.peek().cap <= w) {
            byProfit.push(byCapital.pop().prof);
        }
        if (byProfit.size() === 0) break; // ఏదీ afford చేయలేం → ఆగు
        w += byProfit.pop(); // అత్యధిక-profit project చేపట్టు
    }
    return w;
}
```

Dry Run: `k=2, w=0, profits=[1,2,3], capital=[0,1,1]`

```
byCapital (min on cap): {cap0,prof1}, {cap1,prof2}, {cap1,prof3}
byProfit (max): {}

i=0: w=0
  unlock: peek cap0 ≤ 0 → pop {cap0,prof1}, byProfit push 1
          peek cap1 ≤ 0? లేదు → అగు. byProfit = {1}
  pop max = 1 → w = 0 + 1 = 1
i=1: w=1
  unlock: peek cap1 ≤ 1 → pop {cap1,prof2}, push 2
          peek cap1 ≤ 1 → pop {cap1,prof3}, push 3. byProfit = {2,3}
  pop max = 3 → w = 1 + 3 = 4
return 4 ✓
```

Complexity:

- **Time:** $O(n \log n + k \log n)$ – init n pushes $O(n \log n)$; k iterations, ప్రతి project ఎక్కువలో ఎక్కువ ఒకసారి రెండు heaps మధ్య కదులుతుంది → total heap ops $O((n+k) \log n)$.
- **Space:** $O(n)$ – రెండు heaps కలిపి $\leq n$ projects.

గుర్తుంచుకోవాల్సినది: ఇది **"greedy + two heaps"** classic. కీలకం: ఒక heap **eligibility/unlock** (ఇక్కడ capital) ప్రకారం, రెండో heap **selection value** (profit) ప్రకారం. "ప్రతి అడుగులో available వాటిలో best తీసుకో, available set కాలంతో పెరుగుతుంది" అనే నమూనా – task scheduling ("Maximum Performance of a Team" #1383, "Task Scheduler", CPU scheduling) కి కూడా ఇదే. **గమనిక:** profits negative అయితే greedy లో "profit ≤ 0 అయితే అగు" అని జోడించాలి; ఇక్కడ profits positive కాబట్టి అవసరం లేదు.

సాధారణ తప్పులు:

- **ప్రతి iteration లో byCapital ని re-scan చేయడం:** unlock loop ప్రతి project ని ఒకసారే తరలిస్తుంది (while condition capital-heap top మీద). పొరపాటున ప్రతిసారీ మొదటినుండి scan చేస్తే $O(k \cdot n)$ కి తిరిగి పడతావు.
- **byProfit ఖాళీ break మర్చిపోవడం:** capital అన్నిటికీ సరిపోకపోతే (`w` చిన్నది), unlock loop ఏదీ push చేయదు; break లేకపోతే empty heap మీద `pop()` → undefined/crash. `size() == 0` check తప్పనిసరి.
- **MinHeap/MaxHeap తారుమారు:** capital కి **min** (చౌకవి ముందు unlock), profit కి **max** (ఎక్కువ లాభం ముందు). తారుమారు చేస్తే greedy తప్పు.
- `<= w` బదులు `< w`: capital సరిగ్గా `w` కి సమానమైన project ని కూడా afford చేయవచ్చు → `<=` వాడు.

10. Find K Pairs with Smallest Sums (LeetCode #373) – Medium

సమస్య: రెండు **ascending sorted** arrays `nums1`, `nums2` మరియు integer `k` ఇస్తారు. ఒక pair అంటే `(u, v)` – `u ∈ nums1`, `v ∈ nums2`. అతి చిన్న **sum (u+v)** ఉన్న **k pairs** ని return చెయ్యి.

Constraints: $1 \leq \text{nums1.length}, \text{nums2.length} \leq 10^5$; $-10^9 \leq \text{nums}[i] \leq 10^9$; $1 \leq k \leq 10^4$; రెండూ sorted.

ఉదాహరణ:

```
nums1 = [1,7,11], nums2 = [2,4,6], k = 3
→ [[1,2],[1,4],[1,6]] (sums 3,5,7 – అతి చిన్న 3)
nums1 = [1,2], nums2 = [3], k = 3
→ [[1,3],[2,3]] (కేవలం 2 pairs ఉన్నాయి)
```

ఎలా ఆలోచించాలి: అన్ని $m \times n$ pairs generate చేసి sum ప్రకారం sort చేసి మొదటి k తీసుకోవడం – $O(m \cdot n \cdot \log(m \cdot n))$. $10^5 \times 10^5$ pairs → అసాధ్యం. **కీలక insight:** arrays sorted కాబట్టి, **అతి చిన్న sum pair ఖచ్చితంగా** $(\text{nums1}[0], \text{nums2}[0])$. తర్వాతి చిన్నవి – ఇప్పటికే తీసిన pair (i, j) నుండి కుడి/కింది neighbors $(i+1, j)$ లేదా $(i, j+1)$. అంటే – matrix లో top-left నుండి "అలలుగా" విస్తరిస్తున్నట్టు. ఇది **min-heap** కి perfect: heap లో candidate pairs ని sum ప్రకారం ఉంచి, ప్రతిసారీ అతి చిన్న sum ని తీసి, దాని తర్వాతి candidate ని push చెయ్యి. మొత్తం pairs generate చేయకుండా, k సార్లు మాత్రమే pop.

Brute Force / naive: అన్ని pairs తయారు చేసి sort:

```
function kSmallestPairsNaive(nums1, nums2, k) {
  const all = [];
  for (const u of nums1) for (const v of nums2) all.push([u, v]);
  all.sort((a, b) => (a[0] + a[1]) - (b[0] + b[1]));
  return all.slice(0, k);
}
```

- **Time:** $O(m \cdot n \cdot \log(m \cdot n))$ – అన్ని pairs + sort.
- **Space:** $O(m \cdot n)$.
- **ఎందుకు సరిపోదు:** $10^{\{10\}}$ pairs – memory & time రెండూ పేలిపోతాయి. మనకు $k (\leq 10^4)$ pairs మాత్రమే కావాలి – అన్నీ generate చేయడం వృథా. Heap తో $O(k \log k)$.

Optimal Approach: Min-heap on sum. **Init:** nums1 లోని మొదటి $\min(\text{len1}, k)$ indices ని $\text{nums2}[0]$ తో జత చేసి $\text{push}([sum, i, 0])$. ఇవి ప్రతి row యొక్క "మొదటి column". k times: heap నుండి min sum pair (i, j) pop, result కి add; తర్వాత అదే row లో తర్వాతి column $(i, j+1)$ (ఉంటే) push. ఇలా అవసరమైన candidates మాత్రమే heap లోకి వస్తాయి.

Solution (JavaScript):

```
// ↑ heap primer నుండి MinHeap.
function kSmallestPairs(nums1, nums2, k) {
  const res = [];
  if (nums1.length === 0 || nums2.length === 0 || k === 0) return res;

  // heap entry = [sum, i, j] ; sum చిన్నది top
  const heap = new MinHeap((a, b) => a[0] - b[0]);

  // ప్రతి row (nums1[i]) కి మొదటి column (nums2[0]) తో seed
```

```

for (let i = 0; i < Math.min(nums1.length, k); i++) {
    heap.push([nums1[i] + nums2[0], i, 0]);
}

while (res.length < k && heap.size() > 0) {
    const [sum, i, j] = heap.pop(); // ప్రస్తుత అతి చిన్న sum pair
    res.push([nums1[i], nums2[j]]);
    // అదే row లో తర్వాతి column ని candidate గా చేర్చు
    if (j + 1 < nums2.length) {
        heap.push([nums1[i] + nums2[j + 1], i, j + 1]);
    }
}
return res;
}

```

Dry Run: nums1 = [1,7,11], nums2 = [2,4,6], k = 3

```

seed (rows 0,1,2 తో col 0):
    [1+2=3, 0,0], [7+2=9, 1,0], [11+2=13, 2,0]

pop min [3,0,0] → res=[[1,2]] ; push [1+nums2[1]=5, 0,1]
    heap: [5,0,1], [9,1,0], [13,2,0]
pop min [5,0,1] → res=[[1,2],[1,4]] ; push [1+nums2[2]=7, 0,2]
    heap: [7,0,2], [9,1,0], [13,2,0]
pop min [7,0,2] → res=[[1,2],[1,4],[1,6]] ; j+1=3 ≥ len → push లేదు
res.length=3=k → ఆగు
return [[1,2],[1,4],[1,6]] ✓

```

Complexity:

- **Time:** $O(k \log k)$ – heap size ఎప్పుడూ $O(k)$ (seed $\leq k$, ప్రతి pop కి ≤ 1 push); k pops, ఒక్కొక్కటి $O(\log k)$.
- **Space:** $O(k)$ – heap + result.

గుర్తుంచుకోవాల్సినది: "రెండు (లేదా k) sorted lists నుండి top/smallest k combinations" – min-heap తో "అంచుని (frontier) విస్తరించడం". Seed = మొదటి row/list; ప్రతి pop తర్వాత తర్వాతి neighbor మాత్రమే push (అన్నీ కాదు). ఇదే pattern "Merge k Sorted Lists" (#23), "Kth Smallest in Sorted Matrix" (#378), "Ugly Number II" కి. Duplicate pairs రాకుండా – ఒకే direction (column ముందుకు) విస్తరించడం వల్ల ప్రతి pair ఒకసారి వస్తుంది.

సాధారణ తప్పులు:

- **అన్ని neighbors push చేయడం:** ప్రతి pop తర్వాత $(i+1, j)$ మరియు $(i, j+1)$ రెండూ push చేస్తే duplicates వస్తాయి $((i, j))$ ని రెండు దారుల నుండి చేరతావు). Seed = మొత్తం మొదటి column, తర్వాత $(i, j+1)$ మాత్రమే → clean, no duplicates.
- **Seed ని k కి limit చేయకపోవడం:** nums1 చాలా పెద్దదైతే (10^5) అన్ని rows seed చేస్తే heap పెద్దది. $\text{Math.min}(\text{nums1.length}, k)$ rows చాలు (k pairs కి అంతకంటే ఎక్కువ rows అవసరం లేదు).
- **Empty arrays / k పెద్దది:** ఏదైనా array ఖాళీ అయితే $\text{nums2}[0]$ undefined → NaN sum. మొదట్లోనే length check. అలాగే pairs మొత్తం k కంటే తక్కువైతే $\text{heap.size()} > 0$ condition సహజంగా ఆపేస్తుంది.

- **heap లో [i,j] బదులు actual values store చేయడం:** తర్వాతి neighbor కనిపెట్టడానికి **indices (i, j)** అవసరం, కేవలం values కాదు. `[sum, i, j]` store చెయ్యి.

11. Find Median from Data Stream (LeetCode #295) – Hard

సమస్య: ఒక **stream** గా సంఖ్యలు వస్తూ ఉంటాయి. ఒక `MedianFinder` class రాయాలి – రెండు operations: `addNum(num)` (కొత్త సంఖ్యను చేర్చు), `findMedian()` (ఇప్పటిదాకా వచ్చిన అన్నింటి median). Median = sorted order లో మధ్య element (count బేసి), లేదా మధ్య రెండింటి average (count సరి). ఇది **ఎప్పటికప్పుడు (online)** – ప్రతి `addNum` తర్వాత median అడగవచ్చు.

Constraints: $-10^5 \leq \text{num} \leq 10^5$; `findMedian` కి ముందు కనీసం ఒక element; up to 5×10^4 calls.

ఉదాహరణ:

```
addNum(1)      → stream [1]
addNum(2)      → stream [1,2]
findMedian()   → 1.5   ((1+2)/2)
addNum(3)      → stream [1,2,3]
findMedian()   → 2.0   (మధ్య = 2)
```

ఎలా ఆలోచించాలి: ప్రతి `findMedian` కి array ని sort చేస్తే – ప్రతిసారీ $O(n \log n)$, చాలా calls కి భరించలేం. Insertion sorted array లో చేస్తే – search $O(\log n)$ కానీ shift $O(n)$. **కీలక insight:** Median కి మనకు **మొత్తం sorted order అవసరం లేదు** – కేవలం "మధ్య దగ్గర ఉన్న ఒకటి/రెండు elements" చాలు. కాబట్టి stream ని **రెండు సగాలుగా** విడదీద్దాం: **చిన్న సగం** ఒక **MAX-heap** లో (`lo` – దాని top = చిన్న సగంలో అతి పెద్దది = median కి ఎడమ అంచు), **పెద్ద సగం** ఒక **MIN-heap** లో (`hi` – దాని top = పెద్ద సగంలో అతి చిన్నది = median కి కుడి అంచు). రెండు heaps sizes ని balanced గా (తేడా ≤ 1) ఉంచితే – median ఎప్పుడూ ఈ **రెండు tops** దగ్గరే! `addNum` $O(\log n)$, `findMedian` $O(1)$.

Brute Force / naive: ప్రతి `addNum` లో sorted array లో సరైన స్థానంలో insert (binary search + splice). `findMedian` $O(1)$. **Time:** `addNum` $O(n)$ (splice shift). n `addNums` $\rightarrow O(n^2)$. 5×10^4 calls $\rightarrow \sim 2.5 \times 10^9$ shifts, నెమ్మది. Heaps $O(\log n)$ per add కి తగ్గిస్తాయి.

Optimal Approach: రెండు heaps – `lo` (MaxHeap, చిన్న సగం), `hi` (MinHeap, పెద్ద సగం).

Invariants: (1) `lo` లోని ప్రతిదీ $\leq$ `hi` లోని ప్రతిదీ; (2) `lo.size() == hi.size()` లేదా `lo.size() == hi.size() + 1` (`lo` కి ఎప్పుడూ సమానం లేదా ఒకటి ఎక్కువ). **addNum:** ముందు `lo` లో push, వెంటనే `lo` top ని `hi` కి తరలించు (ordering balance), తర్వాత `hi` పెద్దదైతే top ని తిరిగి `lo` కి (size balance).

findMedian: `lo` పెద్దదైతే `lo.peek()`, లేకపోతే $(\text{lo.peek()} + \text{hi.peek()}) / 2$.

Solution (JavaScript):

```
// ↑ heap primer నుండి MinHeap, MaxHeap.
class MedianFinder {
  constructor() {
    this.lo = new MaxHeap(); // చిన్న సగం – top = అతి పెద్దది (median ఎడమ అంచు)
```

```

        this.hi = new MinHeap();    // పెద్ద సగం - top = అతి చిన్నది (median కుడి అంచు)
    }

    addNum(num) {
        this.lo.push(num);           // మొదట చిన్న-సగంలో పెట్టు
        this.hi.push(this.lo.pop()); // lo top ని hi కి → ordering సరిచెయ్యి
        if (this.hi.size() > this.lo.size()) {
            this.lo.push(this.hi.pop()); // size balance: lo ≥ hi గా ఉంచు
        }
    }

    findMedian() {
        if (this.lo.size() > this.hi.size()) {
            return this.lo.peek(); // బేసి total → lo top
        }
        return (this.lo.peek() + this.hi.peek()) / 2; // సరి total → రెండు tops average
    }
}

```

Dry Run: addNum(1), addNum(2), findMedian(), addNum(3), findMedian()

```

addNum(1): lo.push(1) lo{1} ; hi.push(lo.pop()=1) hi{1},lo{} ;
           hi.size 1 > lo.size 0 → lo.push(hi.pop()=1) lo{1},hi{}
addNum(2): lo.push(2) lo{2,1}(max-top 2) ; hi.push(lo.pop()=2) hi{2},lo{1} ;
           hi.size 1 > lo.size 1? లేదు. → lo{1}, hi{2}
findMedian(): lo.size 1 > hi.size 1? లేదు → (lo.peek()+hi.peek())/2 = (1+2)/2 = 1.5 ✓
addNum(3): lo.push(3) lo{3,1} ; hi.push(lo.pop()=3) hi{2,3}(min-top 2),lo{1} ;
           hi.size 2 > lo.size 1 → lo.push(hi.pop()=2) lo{2,1},hi{3}
findMedian(): lo.size 2 > hi.size 1 → lo.peek() = 2 ✓

```

Complexity:

- **addNum:** $O(\log n)$ – constant number of heap push/pop, ఒక్కొక్కటి $O(\log n)$.
- **findMedian:** $O(1)$ – కేవలం రెండు `peek()` .
- **Space:** $O(n)$ – అన్ని elements రెండు heaps లో.

గుర్తుంచుకోవాల్సినది: "Streaming / running median", "median of a window" → **two heaps**

(max-heap చిన్న సగం, min-heap పెద్ద సగం, sizes balanced). గుర్తుంచుకోవాల్సిన addNum ఉపాయం: **"lo లో push → lo top ని hi కి → hi పెద్దదైతే తిరిగి lo కి"** – ఈ మూడు అడుగులు ordering & size రెండింటినీ ఒకేసారి కాపాడతాయి. ఇదే "Sliding Window Median" (#480, + lazy deletion), "IPO", "Sort a stream" లాంటి stream problems కి వున్నాయి.

సాధారణ తప్పులు:

- **MinHeap/MaxHeap తారుమారు:** lo (చిన్న సగం) **max-heap** గా ఉండాలి (median ఎడమ అంచుని $O(1)$ లో ఇవ్వడానికి); hi **min-heap**. తారుమారు చేస్తే tops median కాదు.
- **Balance step మర్చిపోవడం:** కేవలం lo లో push చేసి hi కి తరలించకపోతే, elements sizes/ordering తప్పు అవుతాయి. మూడు అడుగులూ ప్రతి addNum లో తప్పనిసరి.
- **Size convention గందరగోళం:** ఇక్కడ lo ఎప్పుడూ సమానం లేదా ఒకటి ఎక్కువ. కాబట్టి బేసి total లో median = `lo.peek()` . hi ఎక్కువ ఉంచే convention వాడితే findMedian logic మార్చాలి – ఏదో

ఒకటి consistent గా వాడు.

- **Empty heap peek:** findMedian కి ముందు కనీసం ఒక element guaranteed. కానీ generic కోడ్లో empty అయితే `undefined + undefined = NaN` – production లో check జోడించు.

ముగింపు (Summary) – రెండు patterns, ఒక్క వాక్యంలో

- **Binary Search:** "sorted / monotonic / $O(\log n)$ " కనిపిస్తే – search space ని సగం చేస్తూ కుదించు. Off-by-one తప్పించడానికి **lower-bound half-open template** (`lo<hi` , `hi=mid` , `hi=n`) కంఠతా. Array విలువల మీద కాకపోతే **answer/partition మీద** search (Problems 3, 7). Rotated → "ఏ సగం sorted?" (Problems 4, 6).
- **Heap:** "top-K / kth / streaming median / greedy-best-each-step" కనిపిస్తే – heap. JS లో built-in లేదు కాబట్టి **MinHeap/MaxHeap class** కంఠతా. **k-th largest** → **size-k min-heap** (తిరకాను!). **Streaming median** → **two heaps**. **Greedy unlock** → **two heaps** (Problem 9).

రెండూ కలిపి – LeetCode 150 లోని ఈ 11 problems మాత్రమే కాదు, వీటి **variants వందలు** నీకు లొంగుతాయి. Pattern గుర్తుపడితే, code దానంతట అదే వస్తుంది. All the best! 🚀